

CSC 108 – Project #4

Deadline (December 26)

Objective

Investigate a chosen algorithm in Artificial Intelligence (AI) – Machine Learning, analyze its computational complexity, and demonstrate its ability to solve a specific task effectively. This project integrates theoretical understanding, practical implementation, and critical evaluation.

1. Research Paper

A detailed research paper (~2000-3000 words) addressing the following:

- Algorithm Background and Theoretical Foundations: Describe the origins, purpose, and general principles of the algorithm.
- Computational Complexity Analysis: Provide a concise analysis of the algorithm's time and space complexity
- Step-by-Step Explanation: Explain the functioning of the algorithm, including pseudocode or flowcharts.
- Application to the Assigned Task: Detail how the algorithm is applied to solve your group's assigned task. Assigned tasks include:
 - Image and Object Recognition
 - Sentiment Analysis in Text
 - Chatbot Development
 - Customer Segmentation
 - Predicting House Prices
 - Fraud Detection in Transactions
 - Pathfinding for Autonomous Vehicles
 - Recommender System
 - Spam Email Detection
 - Disease Diagnosis
 - Predict Stock Market Trends
 - Language Translation
 - Music Genre Classification
 - Weather Prediction

(Note: Your assigned task can be found at <https://tinyurl.com/bdfssy47>.)

- Critical Evaluation:
Assess the algorithm's strengths, weaknesses, and potential areas for improvement.

2. Implementation

- Develop a working implementation of the algorithm in a programming language of your choice (e.g., Python, Java).
- The implementation must address the assigned task, producing clear and accurate results.
- Provide well-documented code with instructions for testing.

3. Presentation

- Create a 10-minute video presentation that includes:
 - Overview of the research findings.
 - Explanation of the code and demonstration of its performance.
 - Evaluation of the algorithm's suitability for the task.
- The video should be professional, well-structured, and submitted in a common video format (e.g., MP4).

Criteria for Grading

Research Paper (50%)

Criteria	Excellent (9-10)	Good (7-8)	Fair (5-6)	Poor (0-4)
Algorithm Background	Clearly and thoroughly explains the algorithm's purpose, history, and theoretical foundations.	Adequately explains the algorithm's purpose and theory with minor gaps.	Explanation is vague or missing key details about the algorithm.	Lacks explanation or shows significant misunderstanding of the algorithm.
Computational Complexity	Detailed, accurate analysis with examples for best, average, and worst-case complexities.	Covers complexity but with minor errors or lacks examples.	Complexity analysis is incomplete or contains errors.	Missing or significantly incorrect complexity analysis.
Step-by-Step Explanation	Comprehensive explanation with clear diagrams/pseudocode, fully understandable.	Good explanation with diagrams/pseudocode but minor gaps in clarity or completeness.	Basic explanation with insufficient detail or unclear presentation	Poor explanation, lacks clarity, diagrams, or proper understanding.
Application to Task	Thoroughly explains how the algorithm is applied to the task with relevant examples.	Explains the application to the task but lacks depth or clear examples.	Basic explanation of application with limited or unclear examples.	Fails to connect the algorithm to the task meaningfully.
Critical Evaluation	Insightful evaluation of strengths, limitations, and improvements with clear reasoning.	Reasonable evaluation but lacks depth or misses key points.	Evaluation is superficial or lacks meaningful analysis.	Weak or no evaluation of the algorithm's strengths, limitations, or improvements.

Implementation (30%)

Criteria	Excellent (9-10)	Good (7-8)	Fair (5-6)	Poor (0-4)
Correctness and Results	Code runs flawlessly and solves the task accurately with comprehensive testing.	Code solves the task with minor issues or incomplete testing.	Code works partially, with limited testing or major errors in output.	Code fails to produce correct results or is incomplete.
Efficiency	Implementation is optimized and aligns with theoretical complexity.	Code is functional but could be more efficient or has minor inefficiencies.	Code is functional but has significant inefficiencies or diverges from theory.	Code is inefficient or unoptimized for the given task.
Documentation	Well-documented with clear comments and detailed instructions for testing.	Sufficient documentation, but some parts are unclear or lack detail.	Minimal documentation that makes understanding and testing difficult.	Poor or missing documentation, making the code difficult to follow or test.

Presentation (20%)

Criteria	Excellent (9-10)	Good (7-8)	Fair (5-6)	Poor (0-4)
Content and Structure	Covers all required components (overview, findings, demo, evaluation) in a clear, logical structure.	Covers most components but with minor gaps or lack of clarity.	Covers some components but lacks organization or clarity.	Lacks key components or is disorganized and unclear.
Demonstration of Code	Code demonstration is clear, thorough, and effectively highlights functionality and performance.	Demonstration is adequate but misses some details or clarity.	Demonstration is basic or unclear, with limited focus on functionality or performance..	Poor or no demonstration of code, unclear or missing explanations.
Professionalism	Presentation is polished, engaging, and adheres to time limits.	Presentation is clear but less polished or slightly exceeds time limits.	Presentation lacks polish or exceeds time limits significantly.	Poorly delivered or unprofessional presentation.